

Prototype Gap Analysis

roi-v2.html (ad-hoc GPT) vs prototype-v3.html (pm-os pipeline) | May 2026

The Numbers

Metric	roi-v2.html (GPT ad-hoc)	prototype-v3.html (pm-os)
Pages / nav sections	10 (Command Center, Inventory, Governance, Spend, Decisions, Actions, Connectors, Lenses, Approved Answers, Audit)	2 (Workloads table, Cost Intelligence)
JS functions	65	31
Interactive workflows	Decision wizard, Connector setup, Policy creation, Case mgmt, Bulk actions, Asset drawer, ROI drawer, Jira overlay, Onboarding wizard, Demo mode, Persona switching, Amazon Q chat	Filter/sort, Split panel (4 tabs), Card/table toggle, Cost dismiss, Dark mode
Data entities	Workloads, connectors (11), policies, violations, exceptions, cases, audit entries, lenses, approved answers, ROI sources	Workloads (14), cost recommendations
External libraries	Chart.js (3 charts)	None

The gap is roughly **5x in feature surface area**. Both files are ~1,950 lines, but roi-v2 packs dramatically more functionality per line.

Root Cause Analysis

The pipeline didn't fail. It did exactly what it was told. That's the problem.

1. The PRD was a scope funnel, not a scope explorer

The PRD defined 5 capabilities with explicit v1/v2/v3 phasing. The Scope Boundary table drew hard lines: connectors are v2, business outcome correlation is v2, cross-platform discovery is v2. The pipeline respected those boundaries faithfully.

The GPT ad-hoc process had no such table. When GPT proposed a connector page, nobody said "that's v2." The conversation was **divergent** — each iteration could expand the solution space. The pipeline is **convergent** — each stage narrows toward what was defined.

The pipeline lacks a "scope expansion" mechanism. Every stage takes the PRD's scope as gospel. No stage is empowered to say "the PRD scoped this for v2, but a lightweight version belongs in v1 because the prototype needs it to tell a coherent story."

2. The prototype skill builds to spec, not to product vision

Stage 5 translates the Design Spec into working HTML. Design-spec-v3 specified a workload table, split panel, and cost intelligence page. The prototype built exactly that — 26/26 validation checks passed.

In the GPT conversation, the prototype wasn't validating a spec — it was **exploring a product**. The prototype was the thinking medium, not the output artifact.

The pipeline treats the prototype as a verification step ("does this match the spec?") rather than a discovery step ("what else does the product need?").

3. No "product demo narrative" forcing function

roi-v2.html tells a story: command center → inventory → governance → spend → decisions → actions → connectors. It's a product tour. prototype-v3.html is a feature implementation: table + detail panel + cost page. Technically correct but narratively flat.

The pipeline has no stage that asks: "If I had to demo this for 5 minutes, what navigation flow would I walk through?" That question forces building the connective tissue between features.

4. Research underweighted competitor UX patterns

Research analyzed ServiceNow's capabilities (discovery, governance, cost tracking) but not its **interaction patterns** — the stepper workflow, connector marketplace, case management. The GPT conversation absorbed these organically because the user shared competitive screenshots.

5. Missing maximalist-first prototype stage

The ad-hoc process produced a maximalist prototype first, then would scope down. The pipeline produces a minimalist prototype and relies on feedback loops to expand. But feedback loops are designed for **corrections**, not **expansions**.

Six Plugin Improvements to Close the Gap

1. Add a "Vision Prototype" stage

Insert Stage 4.5 or add a two-mode Stage 5. **Vision mode** builds a maximalist, demoable prototype including lightweight v2/v3 features and placeholder pages. **Spec mode** (current behavior) builds the production-faithful version. The vision prototype goes to the eng alignment meeting; the spec prototype goes to engineering.

2. Interaction Pattern Benchmarking in Research

Add a Researcher step that analyzes competitor **UX patterns**, not just capabilities: navigation patterns (sidebar hierarchy), workflow patterns (wizards, steppers), data management patterns (bulk actions, drawers), and integration patterns (connector marketplaces). This gives the Designer concrete interaction vocabulary.

3. Mandatory "5-Min Demo Script" in Design Spec

Force the Designer to write a demo walkthrough. If it reads "open the table, filter, look at detail panel" — that's a signal the product surface is too narrow. A good demo script naturally demands command centers, stepper workflows, and audit trails.

4. Split PRD scope: "Eng v1" vs "Prototype v1"

Add two columns to the Scope Boundary table. **Eng v1** = what gets built to production quality. **Prototype v1** = what gets included in the demo prototype, even as lightweight placeholders. Connectors are v2 for engineering but the prototype shows a connectors page with "coming soon" state.

5. Add a "Product Completeness" Gandalf question

11th question: "If you built only what's in this PRD, would the result feel like a complete product to a first-time user? Or would it feel like a feature inside a product that doesn't exist yet?" This catches the prototype-v3 problem: great feature, doesn't feel like a product.

6. Competitor UX screenshot ingestion in Stage 0.5

Extend the Current State Auditor to accept competitor screenshots/demos. Produce a "Competitor UX Patterns" section alongside the Pain Map, giving downstream stages a richer vocabulary of what "good" looks like.

Summary

The pipeline produced a higher-quality, more **defensible** artifact — every claim traces to research, every feature traces to the PRD, every interaction traces to the design spec. But it produced a

narrower one.

The fix isn't to make the pipeline less disciplined. It's to add a **divergent exploration phase** (vision prototype) before the convergent execution phase (spec prototype), and to enrich upstream stages with interaction pattern analysis and demo narrative requirements that naturally expand the solution surface.

Six plugin changes, all additive. No stages removed, no quality gates weakened. The pipeline keeps its rigor and gains the ad-hoc process's breadth.